

# SNN在线与本地学习算法研究汇总

Tao

摘要

SNN在线与本地学习算法研究汇总

文章汇总

**RTRL: A learning algorithm for continually running fully recurrent neural networks [1];**

**E-prop: A solution to the learning dilemma for recurrent networks of spiking neurons [2];**

**OTTT: Online Training Through Time for Spiking Neural Networks [3];**

**NDOT: NDOT: Neuronal Dynamics-based Online Training for Spiking Neural Networks [4];**

**S-TLLR: S-TLLR: STDP-inspired Temporal Local Learning Rule for Spiking Neural Networks [5];**

**TESS: TESS: A Scalable Temporally and Spatially Local Learning Rule for Spiking Neural Networks [6]**

## I. INTRODUCTION: 研究背景与问题引入

- 1. SNN 与 ANN 的基本差异
- 2. SNN 训练的核心困难：脉冲的不可求导性质
- 3. 时间依赖性与 credit assignment 问题
- 4. 传统 BPTT/STBP 的基本思想与使用的局限性
- 5. 在线学习与本地学习的研究动机

### A. SNN 与 ANN 的基本差异

ANN 使用连续值激活函数：

$$y = f(Wx)$$

SNN 使用事件驱动 spike：

$$s_t = H(u_t - \theta)$$

在 ANN 中，神经元输出通常是一个连续函数，例如 ReLU 或 sigmoid，输出只依赖当前输入。但在 SNN 中，神经元的输出是 spike，即一个离散的0-1事件，由膜电位是否超过阈值决定。

更关键的是，SNN 是一个带状态的动态系统，神经元的膜电位会随时间累积并衰减，因此当前时刻的输出不仅依赖当前输入，还依赖历史输入序列。

这一点使得 SNN 更接近生物神经系统，但也使得训练变得更加复杂。

## B. SNN 训练的核心困难（不可导性）

Spike generation function:

$$s_t = H(u_t - \theta)$$

Heaviside function:

$$H(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

SNN 训练的核心数学障碍。

SNN 的 spike 是由 Heaviside step function 产生的，它是一个离散函数。

这个函数的导数在绝大多数位置都是零，在阈值点又是不可导的，因此无法直接用于梯度下降。

而深度学习的核心优化方法是 gradient descent 梯度下降，因此这构成了 SNN 训练的第一大障碍。

也就是说，我们无法像 ANN 一样直接对 loss function 进行反向传播。

## C. 时间依赖性与 credit assignment 问题

- SNN neuron state:

$$u_t = f(u_{t-1}, s_{t-1}, x_t)$$

- 当前输出依赖：
  - 当前输入  $x_t$
  - 历史膜电位  $u_{t-1}$
  - 历史 spike  $s_{t-1}$
- 问题：credit assignment over time
- 核心困难：
  - 哪个时间步导致当前 loss?
  - 如何将误差分配到过去?

SNN 的第二个核心困难：时间依赖性。

在 SNN 中，神经元是一个递归动态系统，它的状态由上一时刻的状态决定。

因此当前的输出不仅由当前输入决定，还受到过去所有时间步的影响。

这导致一个关键问题：当我们计算当前 loss 时，无法直接判断是哪一个时间步的输入或状态导致了这个误差。

这就是 temporal credit assignment problem，也就是时间维度上的误差分配问题。

这个问题是后续所有方法（BPTT、RTRL、E-prop 等）的核心出发点。

#### D. BPTT / STBP 的基本思想与局限

- 基本思想：
  - 将 SNN 沿时间展开
  - 转换为 deep feedforward graph
- STBP:
  - surrogate gradient 解决 spike 不可导问题
  - 结合 BPTT 进行训练
- 优点：
  - 可以直接使用 gradient descent
  - 训练效果较好
- 局限：
  - 需要存储完整时间序列
  - 非在线更新
  - 不适合硬件实现

这一页介绍目前最标准的解决方案：**BPTT** 和 **STBP**。

BPTT 的核心思想是把 SNN 在时间维度展开成一个深度神经网络，然后使用链式法则进行反向传播。

但由于 spike 不可导的问题，需要引入 STBP，也就是 surrogate gradient 方法，用一个平滑函数近似 spike 的导数，从而使梯度可以传播。

这种方法的优点是训练效果好，并且可以直接使用现有的深度学习优化框架。

但它的核心问题是必须存储整个时间序列的状态，因此内存消耗随时间增长，并且无法进行实时在线更新。

#### E. 研究动机：在线学习与本地学习

- BPTT / STBP 的问题：
  - memory cost scales with time
  - not suitable for streaming input
  - not biologically plausible
- 目标1: Online learning
  - update weights at each timestep
  - no need to store full history
- 目标2: Local learning
  - weight update uses only local information
  - no global backpropagation required
- 本报告主线：

BPTT/STBP → Online Learning → Local Learning

这一页总结整个 **Introduction** 的研究动机。

虽然 BPTT 和 STBP 可以有效训练 SNN，但它们存在明显的局限性，包括内存消耗随时间增长、无法进行在线更新，以及与生物神经系统机制不一致。

因此研究主要发展出两个方向。

第一个方向是 **online learning**，也就是在每个时间步直接更新参数，而不依赖完整历史。

第二个方向是 **local learning**，也就是权重更新只依赖突触局部可获得的信息，而不需要全局误差信号传播。

本报告的后继内容将围绕这一条演化路径展开，从 RTRL E-prop 出发，然后描述 BPTT 和 STBP，逐步介绍在线学习方法以及局部学习规则。

## II. RTRL: 实时循环学习算法

本部分内容概览

- 网络模型与符号约定
- 误差定义与目标函数
- 核心思想：追踪每个权重对每个输出的影响
- 递推公式的详细推导
- 权重更新规则与在线版本
- 教师强制（Teacher Forcing）变体
- 计算复杂度分析

*PPT* 内容：

- **网络模型**： $n$  个单元（神经元）， $m$  个外部输入。
  - $\mathbf{y}(t)$ ： $n$  维网络输出向量（时刻  $t$ ）
  - $\mathbf{x}(t)$ ： $m$  维外部输入向量
  - $\mathbf{z}(t) = [\mathbf{x}(t); \mathbf{y}(t)]$ ：扩展状态
  - $U$ ：网络单元索引集合； $I$ ：输入索引集合
- **动力学**：

$$s_k(t) = \sum_{l \in U \cup I} w_{kl} z_l(t), \quad k \in U \quad (1)$$

$$y_k(t+1) = f_k(s_k(t)) \quad (2)$$

- **误差**：目标  $d_k(t)$ ，误差  $e_k(t) = d_k(t) - y_k(t)$  若  $k \in T(t)$ ，否则 0。
- **目标**：最小化  $J_{\text{total}} = \sum_t J(t)$ ， $J(t) = \frac{1}{2} \sum_k e_k^2(t)$ 。
- **核心量**： $p_{ij}^k(t) \triangleq \frac{\partial y_k(t)}{\partial w_{ij}}$ 。
- **递推**：

$$p_{ij}^k(t+1) = f'_k(s_k(t)) \left[ \sum_{l \in U} w_{kl} p_{ij}^l(t) + \delta_{ik} z_j(t) \right] \quad (3)$$

初始条件  $p_{ij}^k(t_0) = 0$ 。

- **权重更新**： $\Delta w_{ij}(t) = \alpha \sum_{k \in U} e_k(t) p_{ij}^k(t)$ （在线版本）。
- **教师强制**：用目标值替代实际输出，递推中排除被强制单元。
- **复杂度**： $O(n^3 + mn^2)$  存储， $O(n^4)$  计算每步。

讲稿（详细推导）：

a) 1. 网络模型与符号：我们考虑一个完全循环的神经网络，共有  $n$  个单元（神经元）和  $m$  个外部输入线。在离散时间  $t$ ，网络输出向量记为  $\mathbf{y}(t) \in \mathbb{R}^n$ ，外部输入向量记为  $\mathbf{x}(t) \in \mathbb{R}^m$ 。将两者拼接得到扩展向量  $\mathbf{z}(t) = [\mathbf{x}(t); \mathbf{y}(t)] \in \mathbb{R}^{m+n}$ 。我们定义

$U$  为单元索引集合 ( $|U| = n$ )；

$I$  为输入索引集合 ( $|I| = m$ )；

为了统一处理，给每个输入也分配一个索引，

使得  $z_k(t)$  当  $k \in I$  时取  $x_k(t)$ ，当  $k \in U$  时取  $y_k(t)$ 。

网络中的每个单元  $k$  在时刻  $t$  接收到净输入  $s_k(t)$ ，其计算方式为所有扩展输入  $z_l(t)$  乘上对应权重  $w_{kl}$  之和：

$$s_k(t) = \sum_{l \in U \cup I} w_{kl} z_l(t).$$

然后单元的输出由非线性激活函数  $f_k$  作用，在下一时刻产生：

$$y_k(t+1) = f_k(s_k(t)).$$

这里  $f_k$  通常取 sigmoid 或 tanh 函数。注意，外部输入  $x(t)$  并不会立刻影响输出，而是延迟一个时间步。

整个网络的状态由所有单元的膜电位（或激活值）和输入共同决定，权重矩阵  $\mathbf{W}$  是一个  $n \times (m+n)$  的矩阵，包含了所有单元间和输入到单元的连接。为了引入偏置，我们可以在输入中包含一个始终为 1 的常数输入。

b) 2. 误差与目标函数：在监督学习场景中，某些单元在特定时刻被指定为目标值。令：

$T(t)$  为在时刻  $t$  有目标值的单元集合；

目标值为  $d_k(t)$ ；

定义误差  $e_k(t)$ ：

$$e_k(t) = \begin{cases} d_k(t) - y_k(t), & k \in T(t) \\ 0, & \text{否则.} \end{cases}$$

瞬时误差  $J(t) = \frac{1}{2} \sum_{k \in U} e_k^2(t)$ ，总误差  $J_{\text{total}} = \sum_{t=t_0+1}^{t_1} J(t)$ 。我们的目标是通过梯度下降调整权重  $\mathbf{W}$ ，使  $J_{\text{total}}$  最小。

c) 3. 核心思想：跟踪敏感度  $p_{ij}^k$ ：资格迹：

跟踪敏感度  $p_{ij}^k$

RTRL 的关键创新是定义一个三阶量  $p_{ij}^k(t)$ ，它表示时刻  $t$  单元  $k$  的输出  $y_k(t)$  对权重  $w_{ij}$  的偏导数：

$$p_{ij}^k(t) \triangleq \frac{\partial y_k(t)}{\partial w_{ij}}.$$

这个量回答了这样一个问题：如果我们把权重  $w_{ij}$  微调一点点，单元  $k$  在时刻  $t$  的输出会变化多少？

由于网络是循环的， $y_k(t)$  不仅直接依赖于某些权重（如果  $k = i$ ，即权重  $w_{ij}$  直接连到单元  $i$  时），还通过循环连接间接依赖于所有权重——因为过去的输出会影响当前的膜电位，而膜电位又决定当前的输出。因此， $p_{ij}^k(t)$  必须递归计算。

初始时刻  $t_0$ ，我们假设网络的初始状态（如初始膜电位）与所有权重无关，因此：

$$p_{ij}^k(t_0) = 0, \quad \forall i, j, k.$$

d) 4. 递推公式的详细推导（分步拆解）：-

我们的目标是推导  $p_{ij}^k(t+1)$  的递推式。由于  $y_k(t+1) = f_k(s_k(t))$ ，应用链式法则，第一步总是：

$$p_{ij}^k(t+1) = f'_k(s_k(t)) \cdot \frac{\partial s_k(t)}{\partial w_{ij}}. \quad (4.1)$$

因此，核心问题转化为：计算净输入  $s_k(t)$  对权重  $w_{ij}$  的偏导数  $\frac{\partial s_k(t)}{\partial w_{ij}}$ 。

e) 4.1 展开  $s_k(t)$  的表达式：净输入  $s_k(t)$  是所有来源信号乘以对应权重的总和：

$$s_k(t) = \sum_{l \in U \cup I} w_{kl} z_l(t). \quad (4.2)$$

其中  $U$  是网络单元的索引集合， $I$  是外部输入的索引集合。扩展向量  $z_l(t)$  定义为：

$$z_l(t) = \begin{cases} y_l(t), & l \in U, \\ x_l(t), & l \in I. \end{cases}$$

为了更清楚地看到权重对输出的依赖性，我们可以把 (4.2) 拆成两部分：

$$s_k(t) = \sum_{l \in U} w_{kl} y_l(t) + \sum_{l \in I} w_{kl} x_l(t). \quad (4.3)$$

这里的第二项（输入部分）不会引入对权重的隐式依赖，因为  $x_l(t)$  与权重无关。

f) 4.2 对 (4.3) 直接求导——显式依赖与隐式依赖：现在对 (4.3) 关于  $w_{ij}$  求导。注意： $y_l(t)$  本身也是所有权重的函数（因为过去的权重影响了过去的输出），所以必须区分两类依赖：

- 1) 显式依赖（直接）： $s_k(t)$  的表达式里，哪些地方明确写了  $w_{ij}$ ？只有那些系数恰好是  $w_{ij}$  的项。
- 2) 隐式依赖（间接）： $y_l(t)$  包含  $w_{ij}$ ，所以  $\sum w_{kl} y_l(t)$  这一整项也会随  $w_{ij}$  变化。

因此，我们对 (4.3) 求导，得到三项：

$$\frac{\partial s_k(t)}{\partial w_{ij}} = \underbrace{\sum_{l \in U} w_{kl} \frac{\partial y_l(t)}{\partial w_{ij}}}_{\text{隐式依赖：通过循环输出传播}} + \underbrace{\sum_{l \in U} \frac{\partial w_{kl}}{\partial w_{ij}} y_l(t)}_{\text{显式依赖：来自单元连接的系数}} + \underbrace{\sum_{l \in I} \frac{\partial w_{kl}}{\partial w_{ij}} x_l(t)}_{\text{显式依赖：来自输入连接的系数}}. \quad (4.4)$$

g) 4.3 逐一计算三项的具体值：第一项（隐式依赖）：

$$\frac{\partial y_l(t)}{\partial w_{ij}} = p_{ij}^l(t),$$

这正是我们在前一个时间步已经定义好的量。所以第一项为：

$$\sum_{l \in U} w_{kl} p_{ij}^l(t). \quad (4.5)$$

**第二项（显式依赖，来自单元连接）：**我们问：在  $\sum_{l \in U} w_{kl} y_l(t)$  中，哪一项的系数恰好是我们微调的  $w_{ij}$ ？

- 项的通式是  $w_{kl} \cdot y_l(t)$ 。
- 这个系数  $w_{kl}$  等于  $w_{ij}$  的条件是： $k = i$  且  $l = j$ 。
- 因为  $w_{ij}$  的下标中， $i$  是目标神经元， $j$  是源头（源头可能是单元）。
- 所以只有当  $k = i$ （我们正在计算净输入的神经元恰好是权重所在的目标神经元）且  $l = j$ （这个项恰好来自源头  $j$ ）时，导数为  $y_j(t)$ 。
- 其他情况下，导数为 0。

因此：

$$\sum_{l \in U} \frac{\partial w_{kl}}{\partial w_{ij}} y_l(t) = \begin{cases} y_j(t), & \text{如果 } k = i \text{ 且 } j \in U, \\ 0, & \text{否则.} \end{cases} \quad (4.6)$$

**第三项（显式依赖，来自输入连接）：**同理，在  $\sum_{l \in I} w_{kl} x_l(t)$  中，系数  $w_{kl}$  等于  $w_{ij}$  的条件是  $k = i$  且  $l = j$ ，并且此时  $j$  必须是输入（ $j \in I$ ）。导数为  $x_j(t)$ 。

因此：

$$\sum_{l \in I} \frac{\partial w_{kl}}{\partial w_{ij}} x_l(t) = \begin{cases} x_j(t), & \text{如果 } k = i \text{ 且 } j \in I, \\ 0, & \text{否则.} \end{cases} \quad (4.7)$$

**h) 4.4 引入克罗内克函数  $\delta_{ik}$ ：**为了简洁地写出 (4.6) 和 (4.7) 的共同规律，我们引入克罗内克函数  $\delta_{ik}$ ，其定义为：

$$\delta_{ik} = \begin{cases} 1, & \text{如果 } i = k, \\ 0, & \text{如果 } i \neq k. \end{cases}$$

这个函数的作用就像一个“开关”或“门锁”——它只在  $i = k$  时打开，其他时候保持关闭。有了这个记号，我们可以把第二项和第三项合并为：

$$\delta_{ik} \cdot z_j(t). \quad (4.8)$$

为什么可以这样合并？因为：- 如果  $j \in U$ ，则  $z_j(t) = y_j(t)$ ，且第二项非零。- 如果  $j \in I$ ，则  $z_j(t) = x_j(t)$ ，且第三项非零。- 无论哪种情况，其值恰好是  $\delta_{ik} \cdot z_j(t)$ 。

因此，(4.4) 的三项可以合并为：

$$\frac{\partial s_k(t)}{\partial w_{ij}} = \sum_{l \in U} w_{kl} p_{ij}^l(t) + \delta_{ik} z_j(t). \quad (4.9)$$

**i) 4.5 如何理解  $\delta_{ik} z_j(t)$  这一项？：**这一项的含义非常直观：

- 只有当你正在计算的那个神经元  $k$  恰好是权重  $w_{ij}$  所在的神经元  $i$  时（即  $k = i$ ），直接项  $z_j(t)$  才会出现。
- 如果  $k \neq i$ ，则  $s_k(t)$  的表达式中根本不包含  $w_{ij}$  这个系数，所以直接项为 0。
- 而  $\sum_{l \in U} w_{kl} p_{ij}^l(t)$  这一项对所有  $k$  都存在，因为循环连接使得所有单元的历史输出都间接依赖于  $w_{ij}$ 。

所以，完整的影响路径是：

历史影响（通过循环传播，所有人都能感受到）+ 直接影响（仅权重所在的神经元能直接感受到）。

j) 4.6 代回得到最终递推式: 将 (4.9) 代入 (4.1):

$$p_{ij}^k(t+1) = f'_k(s_k(t)) \left[ \sum_{l \in U} w_{kl} p_{ij}^l(t) + \delta_{ik} z_j(t) \right]. \quad (4.10)$$

这就是 **RTRL** 的核心递推方程, 它告诉我们如何从前一个时刻的  $p_{ij}^l(t)$  计算出下一个时刻的  $p_{ij}^k(t+1)$ 。这里  $j$  可以属于  $U$  (单元) 或  $I$  (输入), 而  $i \in U, k \in U$ 。

k) 5. 权重更新规则: 有了  $p_{ij}^k(t)$  之后, 我们可以计算损失  $J(t) = \frac{1}{2} \sum_k e_k^2(t)$  对权重  $w_{ij}$  的梯度:

$$-\frac{\partial J(t)}{\partial w_{ij}} = \sum_{k \in U} e_k(t) \frac{\partial y_k(t)}{\partial w_{ij}} = \sum_{k \in U} e_k(t) p_{ij}^k(t). \quad (5.1)$$

其中误差  $e_k(t)$  在之前已定义。

l) 5.1 离线累积版本: 如果固定权重跑完整个轨迹再更新, 则累积所有时刻的梯度:  
离线累积:

$$\Delta w_{ij} = -\alpha \sum_t \frac{\partial J(t)}{\partial w_{ij}} = \alpha \sum_t \sum_k e_k(t) p_{ij}^k(t). \quad (5.2)$$

m) 5.2 在线实时版本: 如果边跑边更新, 则每个时刻独立更新:  
在线实时:

$$\Delta w_{ij}(t) = \alpha \sum_k e_k(t) p_{ij}^k(t), \quad w_{ij} := w_{ij} + \Delta w_{ij}(t). \quad (5.3)$$

这种在线版本不需要定义训练区间, 网络可以持续运行并学习。

n) 6. 教师强制 (*Teacher Forcing*) 变体: —

在某些任务中, 我们可以把单元  $k$  在时刻  $t$  的输出替换为目标值  $d_k(t)$ , 而不是使用它实际产生的输出  $y_k(t)$ 。这种技巧称为教师强制。

在教师强制下,  $z_k(t)$  的定义变为:

$$z_k(t) = \begin{cases} d_k(t), & \text{如果 } k \in T(t), \\ y_k(t), & \text{如果 } k \in U \setminus T(t), \end{cases}$$

其中  $T(t)$  是在时刻  $t$  有目标值的单元集合。

因为教师值  $d_k(t)$  不依赖于任何权重, 所以  $\frac{\partial d_k(t)}{\partial w_{ij}} = 0$ 。这意味着在递推式中, 被教师强制的单元对应的  $p_{ij}^l(t)$  项必须被移除。因此递推式变为:

$$p_{ij}^k(t+1) = f'_k(s_k(t)) \left[ \sum_{l \in U \setminus T(t)} w_{kl} p_{ij}^l(t) + \delta_{ik} z_j(t) \right]. \quad (6.1)$$

这等价于在计算  $\frac{\partial s_k(t)}{\partial w_{ij}}$  时, 把被教师强制单元的  $\frac{\partial y_l(t)}{\partial w_{ij}}$  视为 0。

o) 7. 计算复杂度分析: 对于全连接网络:

- 有  $n$  个单元,  $m$  个外部输入。
  - 权重总数 =  $n \times (m + n)$ 。
  - 对于每个权重, 需要存储它对所有  $n$  个单元输出的导数  $p_{ij}^k$ 。
- 因此需要存储的变量总数为:

$$\#p = n \times n \times (m + n) = n^3 + mn^2. \quad (7.1)$$

每时间步更新每个  $p_{ij}^k(t+1)$  需要计算:

$$\sum_{l \in U} w_{kl} p_{ij}^l(t),$$

这是一个对  $n$  项的求和, 复杂度为  $O(n)$ 。因此每时间步的总计算复杂度为:

$$O(n) \times O(n^3) = O(n^4). \quad (7.2)$$

这使得 RTRL 在大规模网络中极不实用, 但它是第一个真正意义上的在线梯度算法, 为后续工作 (如 E-prop) 奠定了理论基础。

### III. E-PROP: 资格传播

本部分内容概览

- 脉冲神经网络的一般模型与符号
- 核心数学定理: 梯度分解为学习信号与资格迹的乘积
- 资格迹的定义与递推
- 定理的严格证明 (从 BPTT 到 E-prop 的重新因子化)
- LIF 与 ALIF 神经元的具体资格迹推导
- 学习信号的不同变体 (对称、随机、自适应)
- 奖励基 E-prop (强化学习)

PPT 内容:

- 通用模型: 可观状态  $z_j^t$  (脉冲 0/1), 隐藏状态  $\mathbf{h}_j^t$  (膜电位、适应变量等)。

$$\mathbf{h}_j^t = M(\mathbf{h}_j^{t-1}, z_j^{t-1}, \mathbf{x}^t, \mathbf{W}_j) \quad (4)$$

$$z_j^t = f(\mathbf{h}_j^t) \quad (5)$$

- 核心目标: 计算损失  $E$  对突触权重  $W_{ji}$  的梯度  $\frac{dE}{dW_{ji}}$ 。
- 关键定理 (式 (1)(3)):

$$\frac{dE}{dW_{ji}} = \sum_t L_j^t e_{ji}^t \quad (6)$$

其中  $L_j^t = \frac{dE}{dz_j^t}$  (学习信号),  $e_{ji}^t = \left[ \frac{dz_j^t}{dW_{ji}} \right]_{\text{local}}$  (资格迹)。

- 资格迹定义:

$$e_{ji}^t = \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \cdot \boldsymbol{\epsilon}_{ji}^t \quad (7)$$

$$\epsilon_{ji}^t = \sum_{t' \leq t} \left( \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} \cdots \frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}} \right) \cdot \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}} \quad (8)$$

递推： $\epsilon_{ji}^t = \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} \cdot \epsilon_{ji}^{t-1} + \frac{\partial \mathbf{h}_j^t}{\partial W_{ji}}$

- **定理证明要点：**从 BPTT 经典因子化出发，递归展开  $\frac{dE}{d\mathbf{h}_j^t}$ ，交换求和顺序，得到  $L_j^t$  与  $e_{ji}^t$  的乘积形式。

- **LIF 神经元的资格迹：**

$$e_{ji}^t = \psi_j^t \bar{z}_i^{t-1}, \quad \bar{z}_i^{t-1} = \sum_{\tau} \alpha^{t-1-\tau} z_i^\tau \quad (9)$$

- **ALIF 神经元（适应）：**存在额外慢变量  $a_j^t$ ，资格迹包含慢分量。

$$e_{ji}^t = \psi_j^t (\bar{z}_i^{t-1} - \beta \epsilon_{ji,a}^t) \quad (10)$$

其中  $\epsilon_{ji,a}^{t+1} = \psi_j^t \bar{z}_i^{t-1} + (\rho - \psi_j^t \beta) \epsilon_{ji,a}^t$

- **学习信号变体：**

- 对称： $L_j^t = \sum_k W_{kj}^{out} (y_k^t - y_k^{*,t})$
- 随机： $L_j^t = \sum_k B_{jk} (y_k^t - y_k^{*,t})$ （固定随机）
- 自适应：随机加可塑性

- **奖励基 E-prop：**用于强化学习， $\Delta W_{ji} = -\eta \sum_t \delta^t \mathcal{F}_\gamma(L_j^t e_{ji}^t)$ ， $\delta^t$  为奖励预测误差。

讲稿（详细推导）：

a) 1. 脉冲网络的通用模型：E-prop 面向脉冲循环神经网络（RSNN）。我们考虑一般形式：每个神经元  $j$  具有隐藏状态向量  $\mathbf{h}_j^t$ （可能包含膜电位、适应变量等）和可观输出  $z_j^t \in \{0, 1\}$ （脉冲）。网络动力学由两个函数定义：

$$\mathbf{h}_j^t = M(\mathbf{h}_j^{t-1}, z^{t-1}, \mathbf{x}^t, \mathbf{W}_j), \quad z_j^t = f(\mathbf{h}_j^t).$$

其中  $M$  是状态更新函数（例如 LIF 膜电位更新）， $f$  是脉冲生成函数（通常为 Heaviside 阶跃函数）。权重  $W_{ji}$  是从神经元  $i$  到  $j$  的突触权重。

b) 2. 梯度分解定理：E-prop 的核心数学结果是，损失函数  $E$ （取决于所有脉冲）对权重  $W_{ji}$  的梯度可以表示为：

$$\frac{dE}{dW_{ji}} = \sum_t L_j^t e_{ji}^t,$$

其中  $L_j^t = \frac{dE}{dz_j^t}$  称为“学习信号”， $e_{ji}^t$  称为“资格迹”。资格迹定义为局部可计算的量：

$$e_{ji}^t \triangleq \left[ \frac{dz_j^t}{dW_{ji}} \right]_{\text{local}}.$$

这个局部导数不是近似，它收集了所有可以通过前向计算（仅依赖神经元  $j$  和  $i$  的历史）获得的梯度信息。具体展开为：

$$e_{ji}^t = \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \cdot \epsilon_{ji}^t,$$

其中  $\epsilon_{ji}^t$  是“资格向量”，定义为：

$$\epsilon_{ji}^t = \sum_{t' \leq t} \left( \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} \cdots \frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}} \right) \cdot \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}}.$$

资格向量满足简单的递归方程：

$$\epsilon_{ji}^t = \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} \cdot \epsilon_{ji}^{t-1} + \frac{\partial \mathbf{h}_j^t}{\partial W_{ji}},$$

初始  $\epsilon_{ji}^0 = 0$ 。这个递归完全在前向过程中计算，无需存储历史。

c) 3. 定理的严格证明：我们从 BPTT 的经典因子化开始。将 RNN 按时间展开成前馈网络，每个时间步  $t'$  对应一层，权重共享。对于前馈网络，损失对权重的梯度为：

$$\frac{dE}{dW_{ji}} = \sum_{t'} \frac{dE}{d\mathbf{h}_j^{t'}} \cdot \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}}.$$

这是因为每个时间步的权重副本相同，求和。注意  $\frac{dE}{d\mathbf{h}_j^{t'}}$  是总导数，考虑了后续时刻的影响。

现在递归展开  $\frac{dE}{d\mathbf{h}_j^{t'}}$ 。在计算图节点  $\mathbf{h}_j^{t'}$ ，它连接到  $z_j^{t'}$  和  $\mathbf{h}_j^{t'+1}$ ，因此：

$$\frac{dE}{d\mathbf{h}_j^{t'}} = \frac{dE}{dz_j^{t'}} \frac{\partial z_j^{t'}}{\partial \mathbf{h}_j^{t'}} + \frac{dE}{d\mathbf{h}_j^{t'+1}} \frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}}.$$

定义  $L_j^{t'} = \frac{dE}{dz_j^{t'}}$ ，并继续展开  $\frac{dE}{d\mathbf{h}_j^{t'+1}}$ ，直到最后时间  $T$ （边界为零），得到：

$$\frac{dE}{d\mathbf{h}_j^{t'}} = \sum_{t \geq t'} L_j^t \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \left( \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} \cdots \frac{\partial \mathbf{h}_j^{t'+1}}{\partial \mathbf{h}_j^{t'}} \right).$$

将此代入经典因子化：

$$\frac{dE}{dW_{ji}} = \sum_{t'} \sum_{t \geq t'} L_j^t \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \left( \prod_{\tau=t'+1}^t \frac{\partial \mathbf{h}_j^\tau}{\partial \mathbf{h}_j^{\tau-1}} \right) \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}}.$$

交换求和顺序（先  $t$  后  $t'$ ）：

$$\frac{dE}{dW_{ji}} = \sum_t L_j^t \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \left[ \sum_{t' \leq t} \left( \prod_{\tau=t'+1}^t \frac{\partial \mathbf{h}_j^\tau}{\partial \mathbf{h}_j^{\tau-1}} \right) \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}} \right].$$

方括号中的项正是  $\epsilon_{ji}^t$  的定义，因此得到  $\frac{dE}{dW_{ji}} = \sum_t L_j^t \epsilon_{ji}^t$ 。这完成了证明。

d) 4. LIF 神经元资格迹：对于 LIF 神经元，隐藏状态仅为膜电位  $v_j^t$ ，隐藏状态更新为：

$$v_j^{t+1} = \alpha v_j^t + \sum_i W_{ji} z_i^t - z_j^t v_{th}.$$

忽略重置项对导数的贡献（或按公式处理），我们有  $\frac{\partial v_j^{t+1}}{\partial v_j^t} = \alpha$ ， $\frac{\partial v_j^t}{\partial W_{ji}} = z_i^{t-1}$ （近似）。资格向量变为：

$$\epsilon_{ji}^t = \sum_{t' \leq t} \alpha^{t-t'} z_i^{t'-1} = \bar{z}_i^{t-1},$$

即前突触脉冲的低通滤波。脉冲导数  $\frac{\partial z_j^t}{\partial v_j^t}$  用伪梯度  $\psi_j^t$  代替，故  $\epsilon_{ji}^t = \psi_j^t \bar{z}_i^{t-1}$ 。

e) 5. *ALIF* 神经元资格迹: *ALIF* 神经元有适应变量  $a_j^t$ , 阈值  $A_j^t = v_{th} + \beta a_j^t$ , 适应动态  $a_j^{t+1} = \rho a_j^t + z_j^t$ . 隐藏状态为二维. 资格向量有两个分量, 其中适应分量满足:

$$\epsilon_{ji,a}^{t+1} = \psi_j^t \bar{z}_i^{t-1} + (\rho - \psi_j^t \beta) \epsilon_{ji,a}^t.$$

因为  $\frac{\partial z_j^t}{\partial \mathbf{h}_i^t} = [\psi_j^t, -\beta \psi_j^t]$ , 最终资格迹为:

$$e_{ji}^t = \psi_j^t (\bar{z}_i^{t-1} - \beta \epsilon_{ji,a}^t).$$

这引入了慢分量, 使梯度信息能跨越长时延传播。

f) 6. 学习信号变体: 理想学习信号  $L_j^t = \frac{dE}{dz_j^t}$  需要反向传播. **E-prop** 使用近似:

- 对称:  $L_j^t = \sum_k W_{kj}^{out} (y_k^t - y_k^{*,t})$ , 其中  $W_{kj}^{out}$  是输出权重,  $y_k^t$  是输出神经元的滤波活性. 这要求权重对称。
- 随机:  $L_j^t = \sum_k B_{jk} (y_k^t - y_k^{*,t})$ ,  $B_{jk}$  固定随机, 类似反馈对齐。
- 自适应: 在随机基础上,  $B_{jk}$  通过局部可塑性逐渐演化, 模仿对称情况。

g) 7. 奖励基 *E-prop*: 对于强化学习, 网络输出策略和价值估计. 使用奖励预测误差  $\delta^t = r^t + \gamma V^{t+1} - V^t$ . 权重更新为:

$$\Delta W_{ji} = -\eta \sum_t \delta^t \mathcal{F}_\gamma(L_j^t \bar{e}_{ji}^t),$$

其中  $\mathcal{F}_\gamma$  是折扣因子低通滤波,  $\bar{e}_{ji}^t$  是资格迹的平滑版本. 这使 **E-prop** 适用于深度强化学习任务。

#### IV. RTRL 与 E-PROP 的联系与对比

本部分内容概览

- 数学结构的对比
- 复杂度的对比
- 生物合理性的对比
- 对脉冲神经元的适应性
- 总结与演化关系

*PPT* 内容:

- 数学结构:
  - RTRL:  $\Delta w_{ij}(t) = \alpha \sum_k e_k(t) p_{ij}^k(t)$ ,  $p_{ij}^k$  为三阶张量。
  - E-prop:  $\Delta W_{ji} = -\eta \sum_t L_j^t \bar{e}_{ji}^t$ , 分解为学习信号和资格迹。
  - E-prop 可视为 RTRL 的因子化:  $p_{ij}^k(t)$  被分解为  $L_j^t$  和  $\bar{e}_{ji}^t$  的乘积 (求和)。
- 复杂度:
  - RTRL: 存储  $O(n^3)$ , 计算  $O(n^4)$  (全连接)。
  - E-prop: 存储  $O(S)$  (突触数), 计算  $O(S)$ 。
- 生物合理性:
  - RTRL: 非局部 (需要全局误差和权重矩阵), 不生物。
  - E-prop: 局部资格迹 + 全局学习信号 (类比多巴胺), 更生物合理。
- 适应性:

- **RTRL**: 主要针对连续值单元, 不直接适用于脉冲神经元。
- **E-prop**: 专为脉冲神经元设计, 包含适应神经元的慢变量。
- **总结**: **E-prop** 是 **RTRL** 在脉冲网络中的高效、局部、生物合理的演化版本。

讲稿:

a) 1. 数学结构的对比: **RTRL** 的核心是  $p_{ij}^k(t) = \frac{\partial y_k(t)}{\partial w_{ij}}$ , 它是一个三阶张量, 需要追踪每个权重对每个神经元输出的影响。权重更新需要累加所有输出单元的误差  $e_k(t)$  与相应  $p_{ij}^k$  的乘积。而 **E-prop** 通过分解将这种影响分成两部分: 学习信号  $L_j^t$  (仅依赖于神经元  $j$  的全局影响) 和资格迹  $e_{ji}^t$  (仅依赖突触前后的局部活动)。这样, 原本的  $p_{ij}^k$  被替换为  $L_j^t \cdot e_{ji}^t$  在时间上的求和, 大大简化了存储和计算。

b) 2. 复杂度的对比: **RTRL** 需要存储  $n^3$  量级的变量 (全连接), 计算每步  $O(n^4)$ , 无法扩展。**E-prop** 只需存储每个突触的资格迹 ( $O(S)$ ,  $S$  为突触数), 计算也仅与突触数线性相关, 这是最优的, 因为模拟网络本身就需要  $O(S)$  时间。

c) 3. 生物合理性: **RTRL** 的更新需要每个突触访问所有单元的误差和权重矩阵, 这在大脑中无法实现。**E-prop** 的资格迹仅依赖于前突触和后突触的局部活动 (如脉冲频率和膜电位), 而学习信号可以是全局的 (如多巴胺释放), 这在生物学上有对应。因此 **E-prop** 更符合神经可塑性的约束。

d) 4. 对脉冲神经元的适应性: **RTRL** 最初设计用于连续激活函数, 而脉冲神经元的离散性需要额外的处理 (如伪导数)。**E-prop** 从头针对脉冲设计, 并且自然地包含了 **ALIF** 神经元的慢适应变量, 使其能够解决长时间信用分配问题。**BPTT** 也能处理脉冲网络, 但 **RTRL** 直接用于脉冲网络会面临更大的计算和导数问题。

e) 5. 总结: **E-prop** 可以视为 **RTRL** 在脉冲网络中的高效、局部化、生物合理的演化版本。它继承了 **RTRL** 的在线学习能力, 但通过重新因子化梯度大幅降低了复杂度, 并适配了脉冲神经元的特性。两者共同为循环网络的在线学习提供了理论基础, 其中 **E-prop** 更加实用和生物可信。

## V. 脉冲神经网络反向传播算法: BPTT

$$\frac{\partial L}{\partial \mathbf{W}^l} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \left( \frac{\partial \mathbf{u}^{l+1}[t]}{\partial \mathbf{W}^l} + \sum_{\tau < t} \prod_{i=t-1}^{\tau} \left( \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} + \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]} \right) \frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} \right), \quad (11)$$

[3];

$$\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} + \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]} \quad (12)$$

$$\frac{d\mathbf{u}^{l+1}[i+1]}{d\mathbf{u}^{l+1}[i]} = \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} + \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \cdot \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]}$$

代入 LIF 动力学后展开为:

$$\frac{d\mathbf{u}^{l+1}[i+1]}{d\mathbf{u}^{l+1}[i]} = \lambda \mathbf{I} - \lambda V_{th} \mathbf{I} \cdot \sigma'(\mathbf{u}^{l+1}[i] - V_{th})$$

## 本部分核心结构

1. 前向时空动态与LIF离散方程
2. BPTT完整梯度公式（含颜色标记）
3. 公式拆解：误差传入、当前直连项
4. 公式拆解：时间传播核（泄漏 vs 重置路径）
5. 代理梯度（SG）、STBP严格定义及BPTT固有瓶颈

### PPT 1: 前向时空动态与不可微源头

#### a) PPT内容要点：

- 时空参数：层索引  $l$ ，时间步  $t$ 。
- 神经元状态：膜电位  $u_i^l[t]$ ，脉冲  $s_i^l[t]$ 。
- LIF离散动力学（泄漏 + 硬重置）与脉冲生成（阶跃函数，不可导）：

$$\begin{cases} u_i[t+1] = \lambda(u_i[t] - V_{th}s_i[t]) + \sum_j w_{ij}s_j[t] + b_i, \\ s_i[t+1] = H(u_i[t+1] - V_{th}), \end{cases} \quad (13)$$

#### b) 讲稿：

- 训练时网络沿时间和空间双向展开，构成时空计算图。
- 每个节点由  $(u, s)$  描述，更新遵循LIF方程。
- 方程含两项传播机制：泄漏项  $\lambda u$ （记忆衰减）与重置项  $-V_{th}s$ （硬复位）。
- 脉冲函数  $H$  几乎处处导数为0，此为BPTT需代理梯度的根本原因。

### PPT 2: BPTT完整梯度公式（标准形式）

#### c) PPT内容要点：

- 损失定义：

$$L[t] = \frac{1}{T} \mathcal{L}(\mathbf{s}^N[t], \mathbf{y}), L = \sum_{t=1}^T L[t] \quad (14)$$

- 对权重  $\mathbf{W}^l$  的梯度（论文式(3)）：

$$\frac{\partial L}{\partial \mathbf{W}^l} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \cdot \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \cdot \left( \frac{\partial \mathbf{u}^{l+1}[t]}{\partial \mathbf{W}^l} + \sum_{\tau < t} \left[ \prod_{i=t-1}^{\tau} \left( \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} + \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \cdot \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]} \right) \right] \cdot \frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} \right)$$

#### d) 讲稿：

- 公式总体大结构：外层对  $t$  求和，根据上述的  $L = \sum_{t=1}^T L[t]$ ，内部由“当前的膜电位的贡献”与“历史回溯的膜电位的贡献”相加。
- 红色项  $\frac{\partial s}{\partial u}$ ：脉冲对膜电位的敏感度（不可导，后续用代理梯度替代）。
- 绿色项  $\frac{\partial u[i+1]}{\partial u[i]}$ ：膜电位随着时间自动地进行衰减，等于  $\lambda \mathbf{I}$ （泄漏路径）。
- 青色项  $\frac{\partial u[i+1]}{\partial s[i]}$ ：脉冲对下一时刻膜电位的影响，等于  $-\lambda V_{th} \mathbf{I}$ （重置路径）。
- $\text{CyanPart} \times \text{RedPart} + \text{GreenPart}$  两项在连乘内相加，代表误差沿时间的两条并行传播通道。

**PPT 3:**  $\frac{\partial L}{\partial \mathbf{W}^l}$  的完整传播路径与“当前直连项”

**e) PPT内容要点:** :

- 计算目标: 损失  $L$  对第  $l$  层权重  $\mathbf{W}^l$  的梯度。该权重连接上游层  $l$  (脉冲  $s^l$ ) 与下游层  $l+1$  (膜电位  $u^{l+1}$ );
- 时间下标约定: 严格按公式,  $u^{l+1}[t+1]$  由  $s^l[t]$  驱动。用  $t$  表示当前响应时刻。因此梯度公式中写为  $\frac{\partial u^{l+1}[t]}{\partial \mathbf{W}^l} = s^l[t]$ ;
- 外层求和  $\sum_{t=1}^T$ : 权重在所有时刻共享, 需累加各时刻的梯度贡献。
- 误差传入项  $\frac{\partial L}{\partial s^{l+1}[t]}$ : 由输出层通过空间反向传播 (层间BP) 递推到节点  $(l+1, t)$  的脉冲端。
- 红色代理梯度  $\frac{\partial s}{\partial u}$ : 在同一节点  $(l+1, t)$  内部, 将误差从“脉冲”映射到“膜电位”。
- 当前直连项  $\frac{\partial u^{l+1}[t]}{\partial \mathbf{W}^l} = s^l[t]$ : 空间上的直接传播。由上游节点  $(l, t)$  的脉冲, 通过权重  $\mathbf{W}^l$  直接传到下游节点  $(l+1, t)$  的膜电位。
- 无时间回溯: 当前项只依赖当前时刻输入  $s^l[t]$ , 不含  $\prod$  或  $\sum_{\tau < t}$ , 纯空间部分

**f) 讲稿:** :

- 整个梯度刻画的是: 上游层  $l$  在所有时刻的脉冲, 如何通过权重影响下游层  $l+1$  的膜电位, 并最终改变损失。
- 为统一符号, 论文将层间延迟吸收进时间索引, 故  $\frac{\partial u^{l+1}[t]}{\partial \mathbf{W}^l} = s^l[t]$  在约定下精确成立。
- 当前项的计算路径: 节点  $(l, t)$  脉冲  $\xrightarrow{W^l}$  节点  $(l+1, t)$  膜电位。
- 反向传播时: 高层误差  $\frac{\partial L}{\partial s^{l+1}[t]}$  到达节点  $(l+1, t)$  脉冲端。
- 乘以红色代理梯度, 将误差从脉冲传至同一节点的膜电位。
- 随后该膜电位误差乘以  $\frac{\partial u^{l+1}[t]}{\partial s^l[t]} = W^l$ , 沿空间反向传回上层节点  $(l, t)$  的脉冲端。
- 当前项不含连乘或历史求和, 因为膜电位对权重的直接导数只依赖当前输入脉冲, 不依赖过去状态。
- 整体相乘因子为: 误差项  $\times$  代理梯度  $\times$  当前输入脉冲。

**PPT 4:** 公式拆解 (核心)—— 时间传播核与“相乘、相加”的数学本质

**g) PPT内容要点:** :

- 历史回溯项结构:

$$\sum_{\tau < t} \left[ \prod_{i=t-1}^{\tau} (\lambda \mathbf{I} + (-\lambda V_{th} \mathbf{I}) \cdot \sigma') \right] \cdot \frac{\partial u[\tau]}{\partial W}$$

- 节点传播路径 (时间维度): 同一层  $l+1$  内部, 历史时刻  $\tau$  的膜电位节点  $u[\tau]$ , 通过神经元内部动态, 一步一步传播到当前时刻  $t$  的膜电位节点  $u[t]$ 。
- 足迹项  $\frac{\partial u[\tau]}{\partial W} = s[\tau]$ : 历史时刻上游输入的脉冲。
- 相乘 ( $\prod$ ): 链式法则: 从  $\tau$  到  $t$  必须经过中间所有时刻  $t-1, t-2, \dots, \tau$ , 复合函数嵌套求导, 根据链式法则, 连续相乘。
- 括号内相加 (+): 全微分: 同一个时间步  $i \rightarrow i+1$  膜电位更新包含两项:

$$u[i+1] = \lambda u[i] - \lambda V_{th} s[i] + \dots$$

$u[i+1]$  对  $u[i]$  的依赖有两条并行路径：直接泄漏（绿色）和通过脉冲的间接重置（青色 $\times$ 红色）。两路径同时存在，总导数为二者之和（全微分公式： $\frac{dz}{dx} = \frac{\partial z}{\partial x} + \frac{\partial z}{\partial y} \frac{dy}{dx}$ ）。

- 外层求和（ $\sum_{\tau < t}$ ）：所有历史时刻  $\tau = 1, \dots, t-1$  都是独立的源头，各自通过时序核影响当前，总贡献为各源头之和。
- 青色项精确值（依据本论文公式(2)）：

$$\frac{\partial u[i+1]}{\partial s[i]} = -\lambda V_{th} \mathbf{I}$$

（对线性重置项  $-\lambda V_{th} s[i]$  直接求导）

**h) 讲稿：**

- 历史回溯项计算的是：过去时刻  $\tau$  的输入脉冲，经过下游神经元内部时间演化，对当前时刻  $t$  膜电位的影响。
- 节点路径：同一层内部  $u[\tau] \rightarrow u[\tau+1] \rightarrow \dots \rightarrow u[t]$ 。
- 为什么有连乘（ $\prod$ ）？因为时间推进是顺序发生的，复合函数链式法则要求每一步的局部导数连续相乘。这就是“串联”关系。
- 为什么括号内是相加（ $+$ ）？因为在相邻时间步  $i \rightarrow i+1$  中，膜电位受两种并行物理机制控制：泄漏（绿色）和重置（青色 $\times$ 红色）。二者互不排斥，同时作用。根据全微分原理，总传播系数等于各路径贡献的线性叠加。这就是“并联”关系。
- 青色项严格等于  $-\lambda V_{th}$ 。注意：这不是对阶跃函数求导，而是对线性项  $-\lambda V_{th} s[i]$  求导，自变量是  $s$ ，故直接得常数。
- 外层求和（ $\sum_{\tau < t}$ ）是因为每个历史时刻都是独立的误差源头，总影响是各源头贡献的总和。

#### PPT 5：代理梯度（SG）、STBP严格定义与BPTT固有瓶颈

**i) PPT内容要点：**

- 代理梯度（Surrogate Gradient）的数学形式：因  $\frac{\partial s}{\partial u} = 0$  (a.e.)，反向传播时用光滑函数导数替代。常用形式：

$$\sigma'(x) = \frac{1}{a} \text{sign} \left( |x - V_{th}| < \frac{a}{2} \right) \quad \text{或} \quad \sigma'(x) = \frac{1}{a} \frac{e^{(V_{th}-x)/a}}{(1 + e^{(V_{th}-x)/a})^2}.$$

- 使用范围：代理梯度仅用于反向传播（红色项），前向传播仍为原始阶跃函数，不改变脉冲离散性。
- STBP的严格学术定义：

STBP (Spatio-Temporal Backpropagation)  $\equiv$  BPTT + Surrogate Gradient (SG).

STBP并非独立新算法，而是BPTT在SNN场景下的工程化命名，其核心特征为完整的时间展开配合代理梯度。

- BPTT的固有瓶颈：

– 内存复杂度  $\mathcal{O}(T)$ ：反向传播时必须存储全部时刻的膜电位  $u[t]$  和脉冲  $s[t]$ ，以计算时序核连乘。

- 非流式 (Non-streaming): 必须等整个长度为  $T$  的序列处理完毕后才能计算梯度并更新权重, 无法在线学习。
- 无法适配神经形态硬件: 生物神经系统和Loihi等芯片要求逐时刻实时更新, BPTT的批处理特性与之矛盾。

j) 讲稿: :

- 代理梯度是解决脉冲函数不可导的必要工程近似, 仅反向使用, 不影响前向计算。
- STBP在严格意义上就是BPTT配合代理梯度, 二者数学等价, 只是命名侧重点不同。
- BPTT的核心问题来源于时序核的连乘结构, 必须存储全轨迹, 内存随  $T$  线性增长。
- BPTT需整段序列处理完才能更新, 属于离线批处理, 与生物在线学习机制不兼容。
- OTTT通过丢弃重置耦合路径, 将双向时序回溯简化为前向指数衰减累积, 从而打破  $\mathcal{O}(T)$  内存限制并支持逐时刻更新。
- 理解标准BPTT的这些瓶颈, 是理解OTTT贡献的绝对前提。

总结

- BPTT完整保留泄漏与重置两条时间传播路径。
- 代理梯度唯一负责处理脉冲不可导性。
- 理解BPTT的每一项是理解OTTT改进的前提。

## VI. OTTT 与 NDOT: 针对 SNN 的在线训练方法

本部分核心结构

- 1 BPTT 标准形式与核心瓶颈: 时间展开、代理梯度依赖、 $\mathcal{O}(T)$  内存。
- 2 OTTT 的近似解耦: 丢弃重置路径, 时序核退化为固定  $\lambda^{t-\tau}$ , 实现  $\mathcal{O}(1)$  在线学习。
- 3 NDOT 的物理重参数化: 从连续 LIF 微分方程导出动态比值  $e[t]$ , 替代固定  $\lambda$ 。
- 4 NDOT 完整推导 (Appendix A.1): 动态核  $\mathbf{P}_{k,t}$  的化简与递推公式的严格证明。
- 5 本质区别汇总: 固定衰减 vs. 动态比值、近似 vs. 精确、变量定义对照表。

PPT 1: BPTT 标准形式与核心瓶颈

a) PPT内容要点 (含完整变量定义): :

- 离散 LIF 动力学:

$$\mathbf{u}^l[t+1] = \lambda(\mathbf{u}^l[t] - V_{th}\mathbf{s}^l[t]) + \mathbf{W}^l\mathbf{s}^{l-1}[t+1]$$

其中  $\mathbf{u}^l[t]$  为第  $l$  层  $t$  时刻膜电位,  $\mathbf{s}^l[t]$  为脉冲,  $\lambda$  为泄漏常数,  $V_{th}$  为阈值,  $\mathbf{W}^l$  为待训练权重。

- BPTT 标准梯度公式:

$$\frac{\partial L}{\partial \mathbf{W}^l} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \left( \frac{\partial \mathbf{u}^{l+1}[t]}{\partial \mathbf{W}^l} + \sum_{\tau < t} \prod_{i=t-1}^{\tau} \left( \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} + \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]} \right) \frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} \right)$$

- 各项精确定义：

- 红色项  $\frac{\partial \mathbf{s}}{\partial \mathbf{u}}$ ：脉冲发放函数对膜电位的导数（Heaviside 不可微，需代理梯度 SG）。
- 绿色项  $\frac{\partial \mathbf{u}[i+1]}{\partial \mathbf{u}[i]}$ ：膜电位自传播，严格等于  $\lambda \mathbf{I}$ （泄漏路径）。
- 青色项  $\frac{\partial \mathbf{u}[i+1]}{\partial \mathbf{s}[i]}$ ：脉冲对下一时刻膜电位的直接影响，严格等于  $-\lambda V_{th} \mathbf{I}$ （重置路径）。

- BPTT 两大瓶颈：

- 1) 时序连乘中因含  $\frac{\partial \mathbf{s}}{\partial \mathbf{u}}$ ，必须使用代理梯度（近似，理论不清晰）。
- 2) 需存储全部  $T$  个时刻的  $\mathbf{u}$  和  $\mathbf{s}$  以计算连乘，内存复杂度  $\mathcal{O}(T)$ ，且无法在线更新。

b) 讲稿：：

- 离散 LIF 动力学由泄漏项、重置项、输入项三部分构成。
- BPTT 展开后，梯度包含“当前直接贡献”与“历史回溯贡献”两部分。
- 历史回溯中的连乘每步含两条路径：绿色（泄漏）和青色×红色（重置耦合）。
- 红色项因不可微必须用 SG，是 BPTT 近似性的根源。
- 连乘结构迫使反向传播时必须访问所有历史状态，导致内存随  $T$  线性增长。

PPT 2: OTTT 的近似解耦与递推实现

c) PPT 内容要点（含完整推导）：

- 核心近似：在时序连乘中，不对红色项施加代理梯度，而是直接使用其真实导数（几乎处处为 0）：

$$\frac{\partial \mathbf{u}[i+1]}{\partial \mathbf{s}[i]} \frac{\partial \mathbf{s}[i]}{\partial \mathbf{u}[i]} \approx 0 \implies \prod_{i=\tau}^{t-1} (\lambda \mathbf{I} + 0) = \lambda^{t-\tau} \mathbf{I}$$

- 简化后的梯度公式 [3]：

$$\frac{\partial L}{\partial \mathbf{W}^l} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \left( \sum_{\tau \leq t} \lambda^{t-\tau} \frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} \right)$$

- 定义追踪变量  $\hat{\mathbf{a}}^l[t]$ （突触前活动踪迹）：

$$\hat{\mathbf{a}}^l[t] \triangleq \sum_{\tau=1}^t \lambda^{t-\tau} \mathbf{s}^l[\tau]$$

由于  $\frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} = \mathbf{s}^l[\tau]$ （对权重的直接偏导等于输入脉冲），上式可重写为：

$$\nabla_{\mathbf{W}^l} L = \sum_{t=1}^T \mathbf{g}_{\mathbf{u}^{l+1}}[t] (\hat{\mathbf{a}}^l[t])^\top$$

其中  $\mathbf{g}_{\mathbf{u}^{l+1}}[t] = \left( \frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \right)^\top$  为空间梯度。

- 前向递推（在线计算）：

$$\hat{\mathbf{a}}^l[t] = \lambda \hat{\mathbf{a}}^l[t-1] + \mathbf{s}^l[t]$$

此式仅依赖上一时刻的  $\hat{\mathbf{a}}$ ，无需存储历史轨迹。

- OTTT 简化效果：

- 时序依赖退化为固定指数衰减  $\lambda^{t-\tau}$ 。
- 内存复杂度降至  $\mathcal{O}(1)$ （与  $T$  无关）。
- 红色项仅在空间传播入口处保留（仍用 **SG**），时序回溯中完全消除。

d) **讲稿：**：

- **OTTT** 将连乘中的重置路径直接置零，因为其依赖的代理梯度不可靠。
- 时序核简化为纯泄漏  $\lambda^{t-\tau}$ ，成为固定常数衰减。
- 定义追踪变量  $\hat{a}[t]$  为历史输入脉冲的加权和，权重为  $\lambda^{t-\tau}$ 。
- 利用  $\frac{\partial u[\tau]}{\partial W} = s[\tau]$ ，梯度可分解为空间梯度  $\mathbf{g}$  与追踪变量  $\hat{a}$  的外积。
- $\hat{a}$  可前向递推计算，只需保留上一时刻值，实现常数内存。
- 但固定  $\lambda$  无法反映神经元状态的实际波动，这是 **OTTT** 的精度瓶颈。

**PPT 3: NDOT 的物理重参数化—— $e[t]$  的严格推导**

e) **PPT：**：

- **OTTT 的局限：**固定  $\lambda$  是常数，但神经元膜电位的实际演化受充电、泄漏、重置共同影响，衰减率应动态变化。
- **NDOT 核心观点：**用连续 LIF 微分方程推导出的  $e(t)$ , *representing the continuous temporal dependency* 替代固定  $\lambda$ 。
- **Step 1:** 连续 LIF 方程 [4]:

$$u'(t) = -u(t) + I(t)$$

其中  $I(t)$  为输入电流。

- **Step 2:** 定义连续时序依赖  $e(t)$ （隐函数关系）:

$$e(t) \triangleq \frac{\partial u(t+1)}{\partial u(t)} = \frac{u'(t+1)}{u'(t)}$$

推导依据：由隐函数定理， $u(t+1)$  对  $u(t)$  的偏导等于两者对时间导数之比。

- **Step 3:** 代入 LIF 方程消去  $I(t)$ :

$$e(t) = \frac{-u(t+1) + I(t+1)}{-u(t) + I(t)} = \frac{u(t+1) - I(t+1)}{u(t) - I(t)}$$

- **Step 4:** 离散化并代入离散 LIF 方程：在离散时间步  $t$ ， $I[t]$  满足  $u[t+1] = \lambda(u[t] - V_{th}s[t]) + I[t+1]$ 。因此  $u[t+1] - I[t+1] = \lambda(u[t] - V_{th}s[t])$ 。同时由前一时刻方程  $I[t] = u[t] - \lambda^{-1}u[t-1]$ ，代入分母化简得：

$$\mathbf{e}^l[t] = \frac{\mathbf{u}^l[t] - V_{th}\mathbf{s}^l[t]}{\mathbf{u}^l[t-1] - V_{th}\mathbf{s}^l[t-1]}$$

注意：此式完全不包含代理梯度，纯为前向可计算的动态比值。

- **变量定义：** $\mathbf{e}^l[t]$  表示第  $l$  层从时刻  $t-1$  到  $t$  的膜电位状态缩放因子，是 **NDOT** 的核心时序核。

f) 讲稿：：

- OTTT 的固定  $\lambda$  是常数，无法适应膜电位的实际波动。
- NDOT 从连续 LIF 微分方程出发，利用隐函数关系定义动态敏感度  $e(t)$ 。
- 通过代入连续方程消去输入电流，再离散化并代入离散 LIF 方程，最终得到闭式解  $e[t]$ 。
- $e[t]$  是相邻时刻“重置后膜电位”的比值，完全由前向状态决定。
- 该推导全程无需任何代理梯度，因此比 OTTT 的固定衰减更忠实于神经元物理。

PPT 4: NDOT 完整梯度推导（严格依据 Appendix A.1）

g) PPT 内容要点（含完整推导链）：：

- Step 1: 定义动态时序传播核  $\mathbf{P}_{k,t}^l$  (NDOT 论文式(9)):

$$\mathbf{P}_{k,t}^l \triangleq \prod_{i=k}^{t-1} \mathbf{e}^l[i] = \mathbf{e}^l[k] \odot \mathbf{e}^l[k+1] \odot \cdots \odot \mathbf{e}^l[t-1]$$

其中  $\odot$  为逐元素乘法（保持维度匹配）。

- Step 2: 代入  $e[i]$  并执行化简:

$$\begin{aligned} \mathbf{P}_{k,t}^l &= \frac{u[k] - V_{th}s[k]}{u[k-1] - V_{th}s[k-1]} \odot \frac{u[k+1] - V_{th}s[k+1]}{u[k] - V_{th}s[k]} \odot \cdots \odot \frac{u[t-1] - V_{th}s[t-1]}{u[t-2] - V_{th}s[t-2]} \\ &= \frac{\mathbf{u}^l[t-1] - V_{th}\mathbf{s}^l[t-1]}{\mathbf{u}^l[k-1] - V_{th}\mathbf{s}^l[k-1]} \end{aligned}$$

中间项全部抵消，仅剩首尾比值。

- Step 3: 定义时序追踪变量  $\hat{\mathbf{a}}^{l-1}[t]$  (NDOT 论文式(11)):

$$\hat{\mathbf{a}}^{l-1}[t] \triangleq \mathbf{s}^{l-1}[t] + \sum_{k < t} \mathbf{P}_{k,t}^{l-1} \odot \mathbf{s}^{l-1}[k]$$

该变量累加所有历史时刻  $k$  的输入脉冲，经动态核  $\mathbf{P}_{k,t}$  缩放后求和。

- Step 4: 推导相邻时刻递推关系 (Appendix A.1 核心证明): 由定义展开  $\hat{\mathbf{a}}[t-1]$ :

$$\hat{\mathbf{a}}[t-1] = \mathbf{s}[t-1] + \sum_{k < t-1} \mathbf{P}_{k,t-1} \odot \mathbf{s}[k]$$

利用  $\mathbf{P}_{k,t} = \mathbf{e}[t-1] \odot \mathbf{P}_{k,t-1}$  (核递推性质):

$$\begin{aligned} \mathbf{e}[t-1] \odot \hat{\mathbf{a}}[t-1] &= \mathbf{e}[t-1] \odot \mathbf{s}[t-1] + \sum_{k < t-1} \mathbf{P}_{k,t} \odot \mathbf{s}[k] \\ &= \mathbf{P}_{t-1,t} \odot \mathbf{s}[t-1] + \sum_{k < t-1} \mathbf{P}_{k,t} \odot \mathbf{s}[k] \\ &= \sum_{k < t} \mathbf{P}_{k,t} \odot \mathbf{s}[k] \end{aligned}$$

因此:

$$\hat{\mathbf{a}}^l[t] = \mathbf{e}^l[t-1] \odot \hat{\mathbf{a}}^l[t-1] + \mathbf{s}^l[t]$$

• **Step 5: 完整梯度公式** (NDOT 论文式(10)):

$$\nabla_{\mathbf{w}^l} \mathcal{L} = \sum_{t=1}^T \mathbf{g}_{\mathbf{u}^l}[t] (\hat{\mathbf{a}}^{l-1}[t])^\top$$

其中  $\mathbf{g}_{\mathbf{u}^l}[t] = \left( \frac{\partial \mathcal{L}}{\partial \mathbf{s}^l[t]} \frac{\partial \mathbf{s}^l[t]}{\partial \mathbf{u}^l[t]} \right)^\top$  为空间梯度 (仍含 SG)。

**h) 讲稿:**

- NDOT 定义动态核  $\mathbf{P}_{k,t}$  为  $\mathbf{e}$  的连乘, 替代 OTTT 的  $\lambda^{t-\tau}$ 。
- 因  $\mathbf{e}$  是分式比值, 连乘形成效应, 化简为首尾比值, 极大简化计算。
- 定义追踪变量  $\hat{a}[t]$  为所有历史脉冲经动态核缩放后的总和。
- 利用  $\mathbf{P}_{k,t}$  的递推性质, 严格推导出  $\hat{a}[t] = \mathbf{e}[t-1] \odot \hat{a}[t-1] + s[t]$ 。
- 该递推式与 OTTT 形式一致, 但核心系数从固定  $\lambda$  换为动态  $\mathbf{e}[t-1]$ 。
- 最终梯度分解为空间梯度  $\mathbf{g}$  与追踪变量  $\hat{a}$  的外积, 实现  $\mathcal{O}(1)$  在线计算。

**PPT 5: OTTT 与 NDOT 本质区别汇总**

**i) PPT 内容要点 (单条目对照):**

- **时序核 (单步):**
  - OTTT:  $\epsilon[i] = \lambda$ , 固定常数。
  - NDOT:  $\mathbf{e}[i] = \frac{u[i] - V_{th}s[i]}{u[i-1] - V_{th}s[i-1]}$ , 动态状态比值。
- **历史传播核:**
  - OTTT:  $\lambda^{t-k}$ , 固定指数衰减。
  - NDOT:  $\mathbf{P}_{k,t} = \prod_{i=k}^{t-1} \mathbf{e}[i] = \frac{u[t-1] - V_{th}s[t-1]}{u[k-1] - V_{th}s[k-1]}$ , 动态化简。
- **追踪变量递推:**
  - OTTT:  $\hat{a}[t] = \lambda \hat{a}[t-1] + s[t]$ 。
  - NDOT:  $\hat{a}[t] = \mathbf{e}[t-1] \odot \hat{a}[t-1] + s[t]$ 。
- **额外存储:**
  - OTTT: 无。
  - NDOT: 需存  $u[t] - V_{th}s[t]$  以计算  $\mathbf{e}[t]$ 。
- **方法论本质:**
  - OTTT: 对 BPTT 做近似删除 (丢弃重置路径), 时序退化为固定衰减。
  - NDOT: 对 BPTT 做物理重参数化 (用连续动力学比值  $e$  替代离散近似  $\epsilon$ ), 时序核变为动态状态比值。

**j) 共同基础:**

- 均实现  $\mathcal{O}(1)$  内存与在线更新。
- 梯度最终形式均为  $\nabla L[t] = \mathbf{g}[t](\hat{a}[t])^\top$ 。
- 空间传播入口处均保留红色项  $\frac{\partial s}{\partial u}$ , 使用代理梯度。

k) 讲稿 (提纲式): :

- OTTT 和 NDOT 都解决了 BPTT 的内存瓶颈，但路径不同。
- OTTT 删除重置路径，用固定  $\lambda$  做指数衰减，简单但损失时序精度。
- NDOT 用连续物理方程导出的动态  $e[t]$  替代  $\lambda$ ，保留完整时序信息，且利用望远镜化简不增加计算负担。
- 递推形式完全一致，仅固定系数换为动态系数。
- NDOT 是 OTTT 的精度升级版，相同内存成本，更准确的梯度估计。
- 核心差异一句话：OTTT 是“删近似”，NDOT 是“用精确物理量替代近似”。

## VII. S-TLLR 与 TESS：时间本地化与空间本地化学习

本部分核心结构

### 1 背景：BPTT 的瓶颈与三因子学习规则的机遇

### 2 S-TLLR：时间本地化学习规则——广义 STDP 资格迹、次级激活函数 $\Psi$ 、因果/非因果融合

### 3 TESS：时空全本地化学习规则——从 $\mathbf{tr}$ 到 $\mathbf{q}/\mathbf{h}$ 的实例化、本地学习信号生成 (LSG)

### 4 资格迹的进化：从抽象 $\mathbf{tr}$ 到显式张量 $\mathbf{q}$ 与 $\mathbf{h}$

### 5 核心算法对比总结——公式对照、复杂度差异、本地化维度进化

PPT 1: 背景——BPTT 的瓶颈与三因子学习规则的机遇

a) PPT 内容要点: :

- BPTT 的计算瓶颈:

$$\frac{d\mathcal{L}}{d\mathbf{W}^{(l)}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{u}^{(l)}[t]} \frac{\partial \mathbf{u}^{(l)}[t]}{\partial \mathbf{W}^{(l)}}$$

内存复杂度  $O(TLn)$ ，时间复杂度  $O(TLn^2)$ ，随  $T$  线性增长

- 三因子学习规则 (3F) 的一般形式:

$$\Delta w_{ij} = \sum_t \delta_i[t] \cdot e_{ij}[t]$$

其中  $e_{ij}[t]$  为资格迹编码突触前/后活动历史，这个部分为时间信号

$\delta_i[t]$  为学习信号解决空间信用分配

- 标准资格迹的瓶颈:

$$e_{ij}[t] = \beta e_{ij}[t-1] + f(y_i[t])g(x_j[t])$$

内存复杂度  $O(n^2)$ ，不适用于深度卷积 SNN。

- STDP 的启示：同时利用因果（前→后）和非因果（后→前）时序关系

b) 讲稿 (提纲式): :

- BPTT 的时间展开机制导致内存和计算成本随  $T$  线性增长，不适合边缘设备。
- 三因子学习规则提供生物学合理的替代方案，但标准资格迹的内存复杂度为  $O(n^2)$ 。
- STDP 同时利用因果和非因果时序关系，这启发了 S-TLLR 的设计。
- S-TLLR 和 TESS 分别从“时间本地”和“时空全本地”两个层面解决上述问题。

## PPT 2: S-TLLR 的完整推导与时间本地化实现

### c) PPT内容要点 (S-TLLR 核心算法公式): :

#### • 广义 STDP 资格迹 [5]:

$$e_{ij}[t] = \alpha_{pre} \Psi(u_i[t]) \sum_{t'=0}^t \lambda_{pre}^{t-t'} x_j[t'] + \alpha_{post} x_j[t] \sum_{t'=0}^{t-1} \lambda_{post}^{t-t'} \Psi(u_i[t'])$$

- 红色项 (因果): 当前后突触活动  $\Psi(u_i[t]) \times$  前突触历史踪迹
- 蓝色项 (非因果): 当前前突触脉冲  $x_j[t] \times$  后突触历史踪迹

#### • 次级激活函数 $\Psi$ 的作用 (论文附录):

- $\Psi$  是一个以阈值为中心的钟形/三角形函数。
- 衡量“神经元离发放脉冲还有多远”，即使神经元未发放也能产生非零信号。
- 典型形式: 三角形  $\Psi(u) = 0.3 \times \max(1 - |u - v_{th}|, 0)$ 。
- 与代理梯度 (SG) 的区别: SG 用于反向传播 (解决  $\frac{\partial s}{\partial u}$  不可导),  $\Psi$  用于前向资格迹计算 (衡量膜电位接近阈值的程度)。

#### • 前向踪迹递推 (时间本地化关键):

$$\text{tr}(x_j)[t] = \lambda_{pre} \text{tr}(x_j)[t-1] + x_j[t]$$

$$\text{tr}(\Psi(u_i))[t] = \lambda_{post} \text{tr}(\Psi(u_i))[t-1] + \Psi(u_i[t-1])$$

#### • 前向形式的资格迹 [5]:

$$e_{ij}[t] = \alpha_{pre} \Psi(u_i[t]) \text{tr}(x_j)[t] + \alpha_{post} x_j[t] (\text{tr}(\Psi(u_i))[t] - \Psi(u_i[t]))$$

#### • 学习信号与权重更新 [5]:

$$\delta_i^{(l)}[t] = \begin{cases} \frac{\partial \mathcal{L}(\mathbf{y}^L[t]; \mathbf{y}^*)}{\partial y_i^{(l)}[t]} & t \geq T_l \\ 0 & \text{otherwise} \end{cases}$$

$$w_{ij} := w_{ij} + \rho \sum_{t=T_l}^T \delta_i[t] e_{ij}[t]$$

(学习信号通过 BP 或 DFA 生成——时间本地, 但空间非本地)

#### • 复杂度: 内存 $O(Ln)$ , 时间 $O(Ln^2)$ 。

### d) 讲稿 (提纲式): :

- S-TLLR 的资格迹包含两项: 因果项 (红色, 前 $\rightarrow$ 后) 和非因果项 (蓝色, 后 $\rightarrow$ 前)。
- 次级激活函数  $\Psi$  与代理梯度不同——它在前向计算中衡量膜电位的“发放潜力”, 即使神经元沉默也能产生学习信号。
- 通过踪迹变量的前向递推, 资格迹可在当前时刻完全计算, 无需存储历史状态。
- 学习信号  $\delta$  仍来自 BP 或 DFA, 实现时间本地但空间非本地。
- $\alpha_{post}$  控制非因果项:  $-1$  惩罚非因果,  $+1$  奖励同步性,  $0$  排除。
- 内存复杂度  $O(Ln)$ , 证明了“时间本地学习”的可行性。

### PPT 3: TESS 的时空全本地化实现与资格迹进化

#### e) PPT内容要点 (TESS 核心算法公式): :

- 使用显式张量  $\mathbf{q}$  与  $\mathbf{h}$ 
  - S-TLLR 的  $\text{tr}(x_j) \rightarrow \text{TESS}$  的  $\mathbf{q}^{(l)}[t]$  (前突触踪迹张量)
  - S-TLLR 的  $\text{tr}(\Psi(u_i)) \rightarrow \text{TESS}$  的  $\mathbf{h}^{(l)}[t]$  (后突触踪迹张量)

- 前突触踪迹  $\mathbf{q}$  ([6](7)):

$$\mathbf{q}^{(l)}[t] = \lambda_{pre} \mathbf{q}^{(l)}[t-1] + \mathbf{o}^{(l-1)}[t]$$

存储输入历史, 维度  $n^{(l-1)}$ 。

- 后突触踪迹  $\mathbf{h}$  ([6](9)):

$$\mathbf{h}^{(l)}[t] = \lambda_{post} \mathbf{h}^{(l)}[t-1] + \Psi(\mathbf{u}^{(l)}[t-1])$$

存储膜电位活跃历史, 维度  $n^{(l)}$ 。

- 资格迹的向量化形式 ([6](8)(10)):

$$\mathbf{e}_{pre}^{(l)}[t] = \alpha_{pre} \Psi(\mathbf{u}^{(l)}[t]) \otimes \mathbf{q}^{(l)}[t]$$

$$\mathbf{e}_{post}^{(l)}[t] = \alpha_{post} \mathbf{h}^{(l)}[t] \otimes \mathbf{o}^{(l-1)}[t]$$

- 进化 2: 学习信号的本地生成——LSG 机制 ([6](11)):

$$\mathbf{v}_m^{(l)}[t] = \mathbf{B}^{(l)\top} (f(\mathbf{B}^{(l)} \mathbf{o}^{(l)}[t]) - \mathbf{y}^*)$$

- $\mathbf{B}^{(l)}$ : 固定二进制矩阵 (列向量为方波函数, 准正交)
- $f(\cdot)$ : softmax (分类) 或 identity (回归)
- 完全在层内生成, 不依赖其他层状态  $\rightarrow$  空间本地化

- 完整权重更新 ([6](12)-(14)):

$$\Delta \mathbf{W}_{pre}^{(l)}[t] = (\mathbf{v}_m^{(l)}[t] \odot \alpha_{pre} \Psi(\mathbf{u}^{(l)}[t])) \otimes \mathbf{q}^{(l)}[t]$$

$$\Delta \mathbf{W}_{post}^{(l)}[t] = (\mathbf{v}_m^{(l)}[t] \odot \alpha_{post} \mathbf{h}^{(l)}[t]) \otimes \mathbf{o}^{(l-1)}[t]$$

$$\Delta \mathbf{W}^{(l)}[t] = \Delta \mathbf{W}_{pre}^{(l)}[t] + \Delta \mathbf{W}_{post}^{(l)}[t]$$

- 复杂度: 内存  $O(Ln)$ , 时间  $O(LCn)$  ( $C$  为类别数,  $C \ll n$ )。

#### f) 讲稿 (提纲式): :

- TESS 将 S-TLLR 中抽象的  $\text{tr}$  函数实例化为两个显式张量:  $\mathbf{q}$  (前突触踪迹) 和  $\mathbf{h}$  (后突触踪迹)。
- 这一实例化使算法能够以标准的张量运算高效运行于 VGG、ResNet 等深度架构。
- 更根本的进化是 LSG 机制: 用固定矩阵  $\mathbf{B}$  直接在当前层输出上生成学习信号, 彻底切断层间依赖。
- $\mathbf{B}$  的列向量为方波函数, 准正交且硬件友好。
- TESS 实现了真正的“时空全本地”学习——时间和空间信用分配均只依赖本地信息。
- 时间复杂度从 S-TLLR 的  $O(Ln^2)$  降至  $O(LCn)$ 。

#### PPT 4: S-TLLR 与 TESS 的本质区别汇总

g) PPT内容要点（核心算法对比）：：区别 1：学习信号的生成方式（最关键）

- S-TLLR（论文式(10)）：

$$\delta_i^{(l)}[t] = \frac{\partial \mathcal{L}}{\partial y_i^{(l)}[t]} \quad (\text{依赖 BP/DFA, 空间非本地})$$

- TESS（[6](11)）：

$$\mathbf{v}_m^{(l)}[t] = \mathbf{B}^{(l)\top} (f(\mathbf{B}^{(l)} \mathbf{o}^{(l)}[t]) - \mathbf{y}^*) \quad (\text{完全层内生成, 空间本地})$$

区别 2：资格迹的形式（抽象 vs. 实例化）

- S-TLLR（标量，[5](9)）：

$$e_{ij}[t] = \alpha_{pre} \Psi(u_i[t]) \text{tr}(x_j)[t] + \alpha_{post} x_j[t] (\text{tr}(\Psi(u_i))[t] - \Psi(u_i[t]))$$

- TESS（向量化，[6](8)(10)）：

$$\mathbf{e}_{pre}^{(l)}[t] = \alpha_{pre} \Psi(\mathbf{u}^{(l)}[t]) \otimes \mathbf{q}^{(l)}[t], \quad \mathbf{e}_{post}^{(l)}[t] = \alpha_{post} \mathbf{h}^{(l)}[t] \otimes \mathbf{o}^{(l-1)}[t]$$

区别 3：时间复杂度

- S-TLLR:  $\text{MAC}_{\text{S-TLLR}} = 3(T - T_l) \sum n^{(l)} n^{(l-1)} = O(Ln^2)$
- TESS:  $\text{MAC}_{\text{TESS}} = (T - t_l) \sum 2 \times n^{(l)} \times C = O(LCn)$

区别 4：本地化维度

- S-TLLR: 仅时间本地（时间信用分配本地化，空间依赖 BP/DFA）
- TESS: 时空全本地（时间 + 空间信用分配均本地化）

h) 讲稿（提纲式）：：

- S-TLLR 和 TESS 的核心区别在于学习信号的生成方式。
- S-TLLR 使用 BP/DFA 生成  $\delta$ ，依赖全局误差信号；TESS 使用 LSG 生成  $\mathbf{v}_m$ ，完全本地。
- 资格迹形式上，S-TLLR 为标量抽象，TESS 为向量化张量。
- 时间复杂度上，TESS 通过 LSG 将  $O(Ln^2)$  降至  $O(LCn)$ 。
- TESS 是 S-TLLR 的自然进化，从“时间本地”到“时空全本地”。

#### PPT 5: 总结——从 S-TLLR 到 TESS 的进化路线图

i) PPT内容要点：：

- 共同基础：
  - 均基于三因子学习规则（资格迹 + 学习信号）
  - 均受 STDP 启发，融合因果与非因果时序关系
  - 均实现  $O(Ln)$  内存复杂度，与时间步数  $T$  无关
  - 均使用次级激活函数  $\Psi$ （但角色略有不同）
- 关键进化：
  - 学习信号：BP/DFA（S-TLLR） $\rightarrow$  LSG（TESS）——从“误差分发”到“目标投射”

- 踪迹表示：抽象  $\text{tr}(\text{S-TLLR}) \rightarrow \text{显式张量 } \mathbf{q}/\mathbf{h}(\text{TESS})$
- 复杂度： $O(Ln^2)(\text{S-TLLR}) \rightarrow O(LCn)(\text{TESS})$
- 本地化维度：仅时间本地  $\rightarrow$  时空全本地

• 方法论本质：

S-TLLR 证明了“时间本地学习”是可行的；TESS 在此基础上用 LSG 替换 BP，实现了“时空全本地学习”，在不增加内存的前提下大幅降低计算复杂度。

• 一句话总结：

S-TLLR 是将 STDP 生物机制融入时间维度的尝试；TESS 通过固定随机投影将误差信号也彻底本地化，实现了完全意义上的时空本地学习，在保持接近 BPTT 精度的同时将计算成本降低 205–661 倍。

j) 讲稿（提纲式）：

- 两篇工作共同构建了从“时间本地”到“时空全本地”的完整技术路线。
- S-TLLR 证明时间本地学习的可行性；TESS 证明全本地学习的可行性。
- 核心进化：LSG 机制用固定随机投影替代 BP，切断了层间依赖。
- 两者均保持  $O(Ln)$  内存，但 TESS 将时间复杂度降至  $O(LCn)$ 。
- 这是一个从“误差分发”到“目标投射”的方法论飞跃。
- 两者共同为边缘设备上的高效 SNN 训练提供了理论基础与算法支撑。

## VIII. 从 BPTT 标准梯度公式到动态规划：严格推导

$$\frac{\partial L}{\partial \mathbf{W}^l} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \left( \frac{\partial \mathbf{u}^{l+1}[t]}{\partial \mathbf{W}^l} + \sum_{\tau < t} \prod_{i=\tau-1}^{\tau} \left( \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} + \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]} \right) \frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} \right), \quad (15)$$

[3];

### A. 符号定义与化简

为了聚焦推导的核心逻辑，我们暂时省略层号上标  $l$  与  $l+1$ ，并定义如下简写：

$$D_i \triangleq \frac{\partial u[i+1]}{\partial u[i]} + \frac{\partial u[i+1]}{\partial s[i]} \cdot \frac{\partial s[i]}{\partial u[i]} \quad (16)$$

$$S_t \triangleq \frac{\partial L}{\partial s[t]} \cdot \frac{\partial s[t]}{\partial u[t]} \quad (17)$$

$$I_\tau \triangleq \frac{\partial u[\tau]}{\partial W} \quad (18)$$

其中：

- $D_i$  是相邻时刻的时间传播系数（包含泄漏和重置两条路径）；
- $S_t$  是当前时刻的空间误差信号（误差从脉冲域传入膜电位域）；
- $I_\tau$  是历史时刻  $\tau$  的直接输入脉冲（ $\frac{\partial u[\tau]}{\partial W}$  对权重的偏导等于输入脉冲）。

利用上述简写，原公式可以简洁地写成：

$$\boxed{\frac{\partial L}{\partial W} = \sum_{t=1}^T S_t \left( I_t + \sum_{\tau < t} \left( \prod_{i=\tau}^{t-1} D_i \right) I_\tau \right)} \quad (19)$$

### B. 第一步：展开双重求和

将式 (19) 中的括号展开：

$$\frac{\partial L}{\partial W} = \underbrace{\sum_{t=1}^T S_t I_t}_{\text{当前贡献}} + \underbrace{\sum_{t=1}^T \sum_{\tau=1}^{t-1} S_t \left( \prod_{i=\tau}^{t-1} D_i \right) I_\tau}_{\text{历史贡献}} \quad (20)$$

式 (20) 的第二项是一个双重求和，其计算复杂度为  $O(T^2)$ 。接下来我们将通过交换求和顺序来重组这一项。

### C. 第二步：交换求和顺序

在式 (20) 的第二项中，内层对  $\tau$  的求和范围为  $1 \leq \tau \leq t-1$ ，外层对  $t$  的求和范围为  $1 \leq t \leq T$ 。我们可以交换这两个求和的顺序。

令  $1 \leq \tau < t \leq T$ ，则：

$$\sum_{t=1}^T \sum_{\tau=1}^{t-1} A_{\tau,t} = \sum_{\tau=1}^{T-1} \sum_{t=\tau+1}^T A_{\tau,t} \quad (21)$$

将此应用于式 (20) 的第二项：

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T S_t I_t + \sum_{\tau=1}^{T-1} \sum_{t=\tau+1}^T S_t \left( \prod_{i=\tau}^{t-1} D_i \right) I_\tau \quad (22)$$

此时，对于每个固定的历史时刻  $\tau$ ，我们将其贡献累加到所有未来的时刻  $t > \tau$ 。

### D. 第三步：定义误差传播变量 $g[t]$

我们定义一个关键的辅助变量  $g[t]$ ，它表示“损失函数对当前时刻膜电位的敏感度”：

$$g[t] \triangleq \frac{\partial L}{\partial u[t]} \quad (23)$$

根据链式法则， $g[t]$  由两部分组成：

- 1) 直接路径：  $u[t] \rightarrow s[t] \rightarrow L$
- 2) 间接路径（通过未来）：  $u[t] \rightarrow u[t+1] \rightarrow \dots \rightarrow L$

因此：

$$g[t] = \frac{\partial L}{\partial s[t]} \frac{\partial s[t]}{\partial u[t]} + \frac{\partial L}{\partial u[t+1]} \frac{\partial u[t+1]}{\partial u[t]} \quad (24)$$

注意到：

- $\frac{\partial L}{\partial s[t]} \frac{\partial s[t]}{\partial u[t]} = S_t$
- $\frac{\partial L}{\partial u[t+1]} = g[t+1]$
- $\frac{\partial u[t+1]}{\partial u[t]} = D_t$  (这里  $D_t$  包含了完整的传播系数，即绿色项 + 青色项  $\times$  红色项)

于是得到反向递推的核心公式：

$$\boxed{g[t] = S_t + g[t+1] \cdot D_t} \quad (25)$$

其中边界条件为：

$$g[T+1] = 0 \quad (26)$$

(因为在最后一个时刻之后没有误差)。

#### E. 第四步：展开 $g[t]$ 验证其与原双重求和等价

现在我们从  $t = T$  开始反向展开  $g[t]$ ，验证它确实捕捉了原公式中的所有历史贡献。

初始条件：

$$g[T+1] = 0 \quad (27)$$

$t = T$  时刻：

$$g[T] = S_T + g[T+1] \cdot D_T = S_T \quad (28)$$

$t = T-1$  时刻：

$$g[T-1] = S_{T-1} + g[T] \cdot D_{T-1} = S_{T-1} + S_T \cdot D_{T-1} \quad (29)$$

$t = T-2$  时刻：

$$\begin{aligned} g[T-2] &= S_{T-2} + g[T-1] \cdot D_{T-2} \\ &= S_{T-2} + (S_{T-1} + S_T \cdot D_{T-1}) \cdot D_{T-2} \\ &= S_{T-2} + S_{T-1} \cdot D_{T-2} + S_T \cdot D_{T-1} \cdot D_{T-2} \end{aligned} \quad (30)$$

观察展开规律，我们可以归纳出一般形式：

$$\boxed{g[t] = \sum_{t'=t}^T S_{t'} \left( \prod_{i=t}^{t'-1} D_i \right)} \quad (31)$$

其中当  $t' = t$  时，空乘积  $\prod_{i=t}^{t'-1} D_i$  规定为 1。

**验证：**将式 (31) 与式 (22) 对比可以发现， $g[t]$  恰好收集了所有未来时刻的误差信号  $S_{t'}$  经过时间传播后对当前时刻  $t$  的累积影响。这正是原双重求和公式中“历史贡献”部分所表达的内容。

### F. 第五步：用 $g[t]$ 表示最终梯度

最终梯度为所有时刻的  $g[t]$  与直接输入脉冲  $I_t$  的乘积之和：

$$\boxed{\frac{\partial L}{\partial W} = \sum_{t=1}^T g[t] \cdot I_t} \quad (32)$$

为什么这样就能得到完整的梯度？

回顾式 (22)，最终梯度由两部分组成：

- 1)  $\sum_{t=1}^T S_t I_t$ ：当前贡献
- 2)  $\sum_{\tau=1}^{T-1} \sum_{t=\tau+1}^T S_t \left( \prod_{i=\tau}^{t-1} D_i \right) I_\tau$ ：历史贡献

当我们计算  $\sum_{t=1}^T g[t] I_t$  时，将式 (31) 代入：

$$\sum_{t=1}^T g[t] I_t = \sum_{t=1}^T \sum_{t'=t}^T S_{t'} \left( \prod_{i=t}^{t'-1} D_i \right) I_t \quad (33)$$

交换求和顺序（令  $\tau = t$ ， $t' = t$  互换为  $t$  在外， $\tau$  在内）：

$$\sum_{t=1}^T g[t] I_t = \sum_{t'=1}^T S_{t'} I_{t'} + \sum_{\tau=1}^{T-1} \sum_{t=\tau+1}^T S_t \left( \prod_{i=\tau}^{t-1} D_i \right) I_\tau \quad (34)$$

这正是式 (22) 的完整展开。因此两者完全等价。

### G. 第六步：复杂度对比

| 形式         | 计算公式                                                                                                   | 计算复杂度    |
|------------|--------------------------------------------------------------------------------------------------------|----------|
| 双重求和（直接展开） | $\sum_{t=1}^T S_t \left( I_t + \sum_{\tau < t} \left( \prod_{i=\tau}^{t-1} D_i \right) I_\tau \right)$ | $O(T^2)$ |
| 动态规划（反向递推） | $g[t] = S_t + g[t+1] \cdot D_t, \quad \frac{\partial L}{\partial W} = \sum_{t=1}^T g[t] \cdot I_t$     | $O(T)$   |

### H. 第七步：完整的算法流程

---

#### Algorithm 1 BPTT 动态规划实现

---

前向传播：记录所有时刻的  $u[t]$ 、 $s[t]$ 、 $S_t$ 、 $I_t$

初始化： $g[T+1] = 0$

**for**  $t = T, T-1, \dots, 1$  **do**

$g[t] = S_t + g[t+1] \cdot D_t$  {反向递推}

**end for**

$\frac{\partial L}{\partial W} = \sum_{t=1}^T g[t] \cdot I_t$  {计算最终梯度}

---

### I. 第八步：多层网络的推广

对于多层网络，需要在空间维度上额外进行层间误差传播。此时  $g^{(l)}[t]$  的递推变为：

$$g^{(l)}[t] = \frac{\partial L}{\partial s^{(l+1)}[t]} \frac{\partial s^{(l+1)}[t]}{\partial u^{(l+1)}[t]} \frac{\partial u^{(l+1)}[t]}{\partial s^{(l)}[t]} + g^{(l)}[t+1] \cdot D_t^{(l)} \quad (35)$$

其中  $\frac{\partial u^{(l+1)}[t]}{\partial s^{(l)}[t]} = W^{(l+1)}$ （即通过突触权重将误差从第  $l+1$  层传回第  $l$  层）。

最终梯度为：

$$\frac{\partial L}{\partial W^{(l)}} = \sum_{t=1}^T g^{(l)}[t] \cdot I_t^{(l)} \quad (36)$$

### J. 总结

本推导从原始 BPTT 标准梯度公式出发，通过以下关键步骤得到了动态规划形式：

- 1) 定义简写符号  $D_i$ 、 $S_t$ 、 $I_\tau$ ，将公式简化为双重求和；
- 2) 展开双重求和并交换求和顺序；
- 3) 定义误差传播变量  $g[t] = \frac{\partial L}{\partial u[t]}$ ；
- 4) 利用链式法则建立反向递推关系  $g[t] = S_t + g[t+1] \cdot D_t$ ；
- 5) 通过展开  $g[t]$  验证其与原公式等价；
- 6) 最终梯度简化为  $\sum_{t=1}^T g[t] \cdot I_t$ 。

最终结果清晰地展示了 BPTT 的本质：从最后时间步反向递推，将原本  $O(T^2)$  的双重求和问题转化为  $O(T)$  的动态规划问题。这个反向递推的过程，正是 OTTT [3]、NDOT [4] 等在线算法试图“近似”或“替代”的目标。

### 参考文献

- [1] R. J. Williams and D. Zipser, “A learning algorithm for continually running fully recurrent neural networks,” *Neural computation*, vol. 1, no. 2, pp. 270–280, 1989.
- [2] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, and W. Maass, “A solution to the learning dilemma for recurrent networks of spiking neurons,” *Nature Communications*, vol. 11, no. 1, p. 3625, Jul. 2020. [Online]. Available: <https://doi.org/10.1038/s41467-020-17236-y>
- [3] M. Xiao, Q. Meng, Z. Zhang, D. He, and Z. Lin, “Online Training Through Time for Spiking Neural Networks,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 20 717–20 730. [Online]. Available: [https://proceedings.neurips.cc/paper\\_files/paper/2022/file/82846e19e6d42ebfd4ace4361def29ae-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2022/file/82846e19e6d42ebfd4ace4361def29ae-Paper-Conference.pdf)
- [4] H. Jiang, G. D. Masi, H. Xiong, and B. Gu, “NDOT: Neuronal Dynamics-based Online Training for Spiking Neural...” [Online]. Available: <https://openreview.net/forum?id=elF0QoBSFV>
- [5] M. P. E. Apolinario and K. Roy, “S-TLLR: STDP-inspired Temporal Local Learning Rule for Spiking Neural Networks,” Oct. 2024, arXiv:2306.15220 [cs]. [Online]. Available: <http://arxiv.org/abs/2306.15220>
- [6] M. P. E. Apolinario, K. Roy, and C. Frenkel, “TESS: A Scalable Temporally and Spatially Local Learning Rule for Spiking Neural Networks,” in *2025 International Joint Conference on Neural Networks (IJCNN)*, Jun. 2025, pp. 1–9, iSSN: 2161-4407. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11227652>